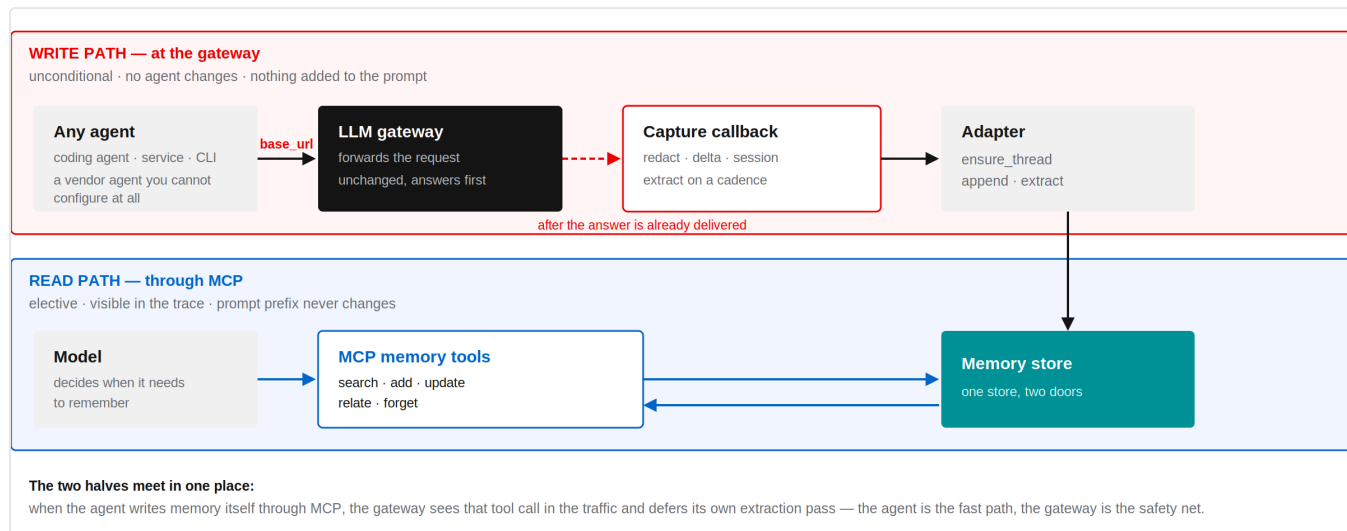


Memory capture at the gateway — readout

Kateryna Romashko · 1 October 2026

A memory system does two jobs: it **writes** — turning conversations into durable facts — and it **reads** — deciding what the model should be reminded of now. This work moves only the *write* job to the LLM gateway. **MCP stays the read path**. The proposal is not the proxy *instead of* MCP; it is the proxy for the half that MCP does badly.



Why the two halves belong in different places

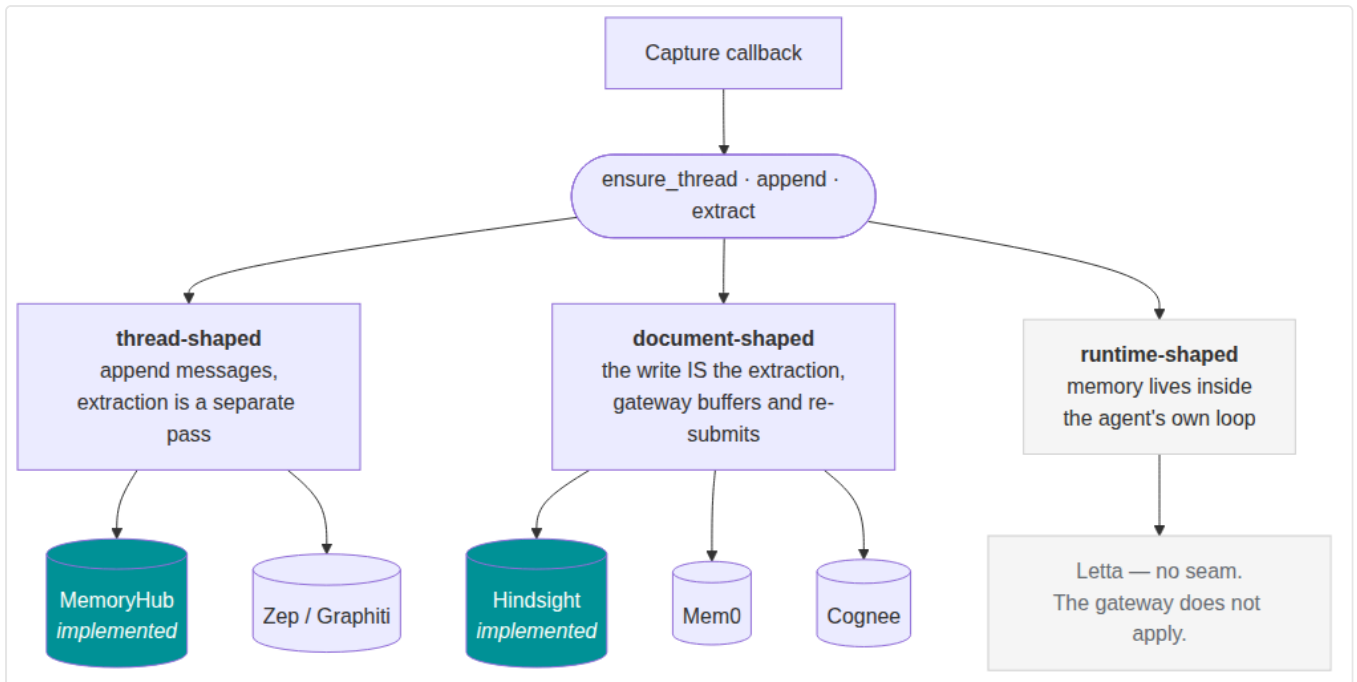
Writing is **unconditional**. Every turn either is or is not captured, and the agent's reasoning does not change that. That is infrastructure work.

Reading is **conditional**. It helps only when memory is actually relevant, and the model is the only thing that knows. A tool call is also visible in the trace, which matters for debugging and for any benchmark.

Today both run through the agent. That makes capture depend on two things: somebody configuring each agent, and the model then choosing to call the tool. Configuration is per agent, per developer, per machine — it cannot be enabled centrally, audited, or noticed when an upgrade removes it. And when the model does not call the tool, the conversation is simply not remembered, and nothing records that it happened.

How a memory system attaches

The gateway talks to a store through three verbs — `ensure_thread`, `append`, `extract`. The cadence of `extract` belongs to the gateway, which is what makes two stores comparable: they are asked to think about the same text at the same moment.



Two adapters are implemented, deliberately on opposite designs: **MemoryHub** (threads, extraction as a separate pass) and **Hindsight** (no threads at all — one document per conversation, extraction inside the write). The rest of the column is the same contract, not code. **Letta is the honest exception**: memory lives inside its agent loop, so there is no seam and the gateway does not apply.

What two full runs showed, 30 September

	Result
Agent-side changes	none. No MCP tool, no prompt instructions, no hooks — only <code>base_url</code>
Duplication	<code>msgs 14 / new 2</code> throughout; <code>new 2</code> again after restarting the gateway mid-conversation
Added latency	none. Extraction took 23–39 s and ran after the answer was delivered; the callback returns in milliseconds
Anthropic endpoint	<code>POST /v1/messages</code> captured . The hook fires; this was the open risk and it is closed
Codex	reached the gateway on <code>/v1/responses</code> , then 500 inside LiteLLM's Ollama parameter mapping (<code>unhashable type: 'dict'</code> on reasoning effort). Not a hook gap — an upstream bug, and probably absent with a hosted model
Credential redaction	0 raw credentials in either store, with the pasted key and token replaced before storage <i>and before hashing</i>
Fact preservation	6/6 facts extracted, 5/6 still current. "API documentation due Friday" was extracted and then retired by a merge at 0.8749
Identity	recorded from the header (<code>kromashk</code> , <code>bob</code>) — but not isolated : one query returned 9 memories across 3 actors in one tenant

The result most useful to the benchmarking work

The same conversation went into MemoryHub and Hindsight at the same moment. They failed in **opposite directions**:

- **MemoryHub over-merges**. It retired a true fact (the Friday deadline) and correctly retired the superseded one (migrate to Go).

- **Hindsight under-merges.** It kept the Friday deadline, and left "migrate to Go" **current alongside** "migrate to Rust" — the contradiction is never resolved, and recall returns both as valid.

No end-to-end recall score shows this: both systems answer "what language?" correctly. It is visible only because one live conversation was fanned out to both stores with identical input. That is the clean way to compare memory systems — running one task twice does not compare them, because memory changes the agent's replies and by the second turn you have two different conversations.

Related, and now reproducible: across 7 reconciliation merges in two runs, the **correct** merge always scored *lower* than at least one false merge (0.8514 and 0.8696 correct, against 0.8819 / 0.8749 / 0.8747 / 0.8671 / 0.8612 false). No threshold separates them.

Why we do not inject memory at the gateway

Injection is where proxy-based memory products get into trouble, and we deliberately do not do it.

- **Prompt caching.** Providers cache on an exact prefix match. A memory block that changes every call invalidates it. Fixable by placing dynamic content last — so this is the weakest of the four objections, not the strongest.
- **Blind retrieval.** The gateway cannot know when memory is needed; it would pay retrieval latency on every call.
- **The loop feeds itself.** Injected memory returns in the next request's history and would be captured as something the user said.
- **Reproducibility.** The agent's author would see a prompt they never wrote — fatal for a measurement harness.

Not done, and known

Scale is unproven: extraction is an LLM call every N turns per session, inside the gateway process; production needs a queue and a worker outside it. Session identity without a header is a fingerprint heuristic with two known failure modes. Redaction filters the *shape* of a credential, not a secret phrased as prose. Retention and deletion of derived data is not built. The runs also found a bug of our own — a running extraction blocked the next callback until LiteLLM cancelled it, silently losing one turn; fixed, with a regression test.

What would help from this call

- Which slice belongs in Milestone 1 and which in Milestone 2.
- Whether to offer shadow mode into the shared benchmarking harness — it is the only way we have found to compare memory systems on identical input.
- Whether the gateway should be the enforcement point for agent identity while that design is still open.